If a failure happens and the application doesn’t respond anymore, you terminate it and start a new instance. If you have a high-load peak, you can spin up more instances of your applications to sustain the increased workload. We say that an application is disposable if it can be started or stopped at any time.

To handle application instances in such a dynamic way, you should design them to start up quickly whenever you need a new instance and gracefully shut down when you don’t need them anymore. A fast startup enables the elasticity of the system, ensuring robustness and resilience. Without a fast startup, you will have performance and availability issues.

A *graceful shutdown* is when an application, on receiving a signal to terminate, stops accepting new requests, completes the ones already in progress, and finally exits. In the case of web processes, that is straightforward. In other cases, such as with worker processes, the jobs they were responsible for must be returned to the work queue, and only afterward can they exit.

2.1.8 Backing services

Backing services can be defined as external resources that an application uses to deliver its functionality. Examples of backing services are databases, message brokers, caching systems, SMTP servers, FTP servers, or RESTful web services. Treating them as attached resources means that you can easily change them without modifying the application code.

Consider how you use databases throughout the software development life cycle. Chances are that you’ll use a different database depending on the stage: development, testing, or production. If you treat the database as an attached resource, you can use a different service depending on the